TrustGuard AI

Explainable Fraud Detection for Digital Banking
with RAG-Based Policy Assistance

Combined Report: Deliverable 1 & Deliverable 2

Problem Definition · Dataset Analysis · Preprocessing Pipeline
Model Architecture · Class Imbalance Handling · Empirical Evaluation

---

Shahzeb Imran 23L-2506

Taha Nawaz 23L-2644

Ali Ahmad 23L-2619

Artificial Intelligence — Spring 2026

Department of Computer Science

National University of Computer and Emerging Sciences, Lahore

April 22, 2026

Contents

Abstract **2**

1 Introduction **3**

2 Problem Definition **3**

2.1 Problem Statement . . . 3

2.2 Target Audience . . . 4

2.3 AI Justification . . . 4

3 Related Work **4**

4 Dataset Description **5**

4.1 Dataset Source . . . 5

4.2 Dataset Schema . . . 5

4.3 Class Distribution . . . 5

4.4 Statistical Summary . . . 6

5 Data Preprocessing Pipeline **6**

5.1 Stage 1: Data Acquisition (`load_dataset.py`) . . . 6

5.2 Stage 2: Basic Inspection (`basic_inspection.py`) . . . 6

5.3 Stage 3: Data Cleaning (`data_cleaning.py`) . . . 7

5.4 Stage 4: Feature Engineering (`feature_engineering.py`) . . . 7

5.5 Stage 5: Feature Scaling (`preprocessing_pipeline.py`) . . . 7

6 Exploratory Data Analysis **7**

6.1 Transaction Volume Over Time . . . 8

6.2 Class Distribution Across Pipeline Stages . . . 8

6.3 Transaction Type Distribution . . . 9

6.4 Fraud Concentration by Transaction Type . . . 9

6.5 Transaction Amount Distribution . . . 10

6.6 Fraud vs. Normal Transaction Amounts . . . 10

6.7 Feature Distributions: Fraud vs. Legitimate . . . 11

6.8 Feature Correlation Heatmap . . . 12

6.9 Feature Importance via Correlation . . . 13

6.10 Top Fraud Patterns . . . 13

7 Key Insights from EDA **14**

8 Class Imbalance Handling **14**
8.1 Step 1: Fraud Simulation Engine (`fraud_simulation_engine.py`) 14
8.2 Step 2: SMOTE (`smote.py`) 15
8.3 Step 3: Class-Weighted Loss Functions 16
8.4 Why This Combination Works 16

9 Model Architecture & Implementation **16**
9.1 Model 1: Logistic Regression (Baseline) — `logistic_regression.py` . . . 17
9.2 Model 2: Random Forest — `random_forest.py` 17
9.3 Model 3: XGBoost — `xgboost_regression.py` 17
9.4 Model 4: Neural Network (MLP) — `neural_networks.py` 18

10 Training & Evaluation **19**
10.1 Validation Strategy 19
10.2 Cross-Validation & Test Results 19
10.3 Model Comparison 20
10.4 Confusion Matrices 21
10.5 ROC and Precision-Recall Curves 22
10.6 Feature Importance 22
10.7 Threshold Optimization 23

11 Error Analysis **23**
11.1 False Positives (FP) — Legitimate Transactions Flagged as Fraud 23
11.2 False Negatives (FN) — Fraudulent Transactions Missed 24
11.3 Key Failure Mode Identified 24

12 Bias Check **24**

13 Contribution Table **26**

14 Conclusion and Future Work **26**

Acknowledgements **27**

Abstract

Financial transactions underpin modern digital life, yet every transfer carries the risk of fraud — a threat costing the global economy trillions of dollars annually. This report presents **TrustGuard AI**, a machine-learning-based fraud detection system built on the PaySim synthetic financial dataset (6.36 million transactions, 0.13% fraud rate). The project spans two deliverables. Deliverable 1 covers formal problem definition, dataset analysis, and a five-stage modular preprocessing pipeline with Exploratory Data Analysis (EDA) revealing key fraud patterns including the balance-drain signature, transaction-type concentration, and extreme class imbalance. Deliverable 2 implements four classifiers — Logistic Regression, Random Forest, XGBoost, and a Multi-Layer Perceptron (MLP Neural Network) — evaluated under 3-Fold Stratified Cross-Validation and held-out test evaluation, supplemented by error analysis and a bias check.

Class imbalance was addressed through a two-step strategy: a custom **Fraud Simulation Engine** that injects realistic synthetic fraud patterns, followed by **SMOTE** (Synthetic Minority Oversampling Technique) to further balance the training set. The Neural Network achieved the best balanced test performance (F1 = 0.69, AUC-ROC = 0.99); XGBoost led on test recall (0.955) and AUC-ROC (0.998). Future phases will incorporate SHAP/LIME explainability and a Retrieval-Augmented Generation (RAG) policy assistant.

Keywords: Financial Fraud Detection, Machine Learning, Explainable AI, RAG Policy Assistant, PaySim, Class Imbalance, SMOTE, Feature Engineering, Data Preprocessing.

1 Introduction

The digital transformation of financial systems has enabled fast and convenient transactions but has also introduced serious challenges in fraud detection. Financial fraud — including unauthorized transfers, account manipulation, and transaction laundering — leads to significant economic losses and erodes trust in banking systems. According to the Association of Certified Fraud Examiners (ACFE), organizations lose approximately 5% of annual revenue to fraud, translating to trillions of dollars globally.

Traditional fraud detection relies on predefined rules and thresholds. While effective for simple cases, these rule-based systems fail to detect complex, evolving fraud patterns and adapt poorly to new strategies. As transaction volumes grow rapidly, especially in emerging markets where digital financial services are expanding and institutional oversight is still maturing, the need for automated and intelligent detection systems becomes essential.

TrustGuard AI is designed to analyze transaction data, identify anomalies, and assist financial institutions in detecting fraudulent activities more effectively. The system is built around a modular preprocessing pipeline, followed by four machine learning classifiers evaluated under rigorous cross-validation. Future phases will incorporate Explainable AI (XAI) using SHAP and LIME to make predictions interpretable, and a Retrieval-Augmented Generation (RAG) policy assistant to provide context-aware regulatory guidance to fraud analysts.

This combined report covers:

- **Deliverable 1:** Problem definition, dataset description, preprocessing pipeline, and EDA.
- **Deliverable 2:** Model architecture, hyperparameter tuning, training, 3-Fold cross-validation, test evaluation, error analysis, and bias check.

2 Problem Definition

2.1 Problem Statement

Financial fraud has become a critical issue with the rapid adoption of digital banking and mobile payment systems. Fraudulent activities such as unauthorized transfers, account manipulation, and transaction laundering result in significant financial losses for both institutions and customers. Traditional rule-based systems are limited in their ability to detect evolving fraud patterns, as they rely on predefined thresholds that can be easily bypassed.

TrustGuard AI is designed to automatically identify suspicious financial transactions using historical transaction data. The system analyzes patterns such as transaction type, amount, and account balance changes to detect anomalies indicating potential fraud. It is also designed to provide explainability for its predictions and integrate a policy-based reasoning component in future phases.

2.2 Target Audience

The system is designed for multiple stakeholders within the financial ecosystem:

- **Banks and Financial Institutions:** Real-time monitoring to reduce financial losses and regulatory risk exposure.
- **Fraud Detection Teams:** Assist analysts in identifying suspicious transactions more efficiently.
- **Regulatory Authorities:** Ensure compliance with PCI-DSS and AML directives.
- **FinTech Companies:** Integrate intelligent fraud detection into mobile and digital banking platforms.

2.3 AI Justification

Fraud detection is characterized by large volumes of transactional data, evolving fraud strategies, and highly imbalanced class distributions. Traditional rule-based systems cannot adapt to new patterns or capture hidden relationships between transaction features. Machine learning provides a more effective solution by learning patterns directly from data — analyzing features such as transaction amount, account balances, and transaction types to identify anomalies with higher accuracy.

Integrating Explainable AI techniques — SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations) — ensures model predictions are interpretable, which is essential in financial systems where transparency and accountability are required by both internal stakeholders and external regulators.

3 Related Work

Dal Pozzolo et al. [2] explored random forests and sampling techniques to address class imbalance in credit-card fraud detection, demonstrating that oversampling minority classes significantly improves recall. Carcillo et al. [3] proposed incremental learning tailored to streaming transaction data, addressing the concept drift inherent in evolving fraud patterns.

López-Vizcaíno et al. [4] evaluated XGBoost and LightGBM on the PaySim dataset,

achieving AUC scores above 0.99 while emphasizing the importance of transaction-type filtering prior to modelling. Samek et al. [5] provide a comprehensive survey of XAI methods applicable to high-stakes domains. Recent work on Retrieval-Augmented Generation [6] has demonstrated that coupling language models with external knowledge bases enables accurate, policy-grounded decision support — an approach adopted in future phases.

4 Dataset Description

4.1 Dataset Source

The dataset used is the PaySim Synthetic Financial Dataset for Fraud Detection [1], obtained from Kaggle. PaySim simulates real-world mobile money transactions by calibrating a financial agent-based model against private transactional data from an African mobile-money service, generating realistic transaction patterns while preserving user privacy. The dataset spans one simulated month across more than six million transactions.

4.2 Dataset Schema

Table 1. *PaySim Dataset Schema*

Column	Type	Description
`step`	int64	Simulated hour of the month (1–743)
`type`	object	Transaction category: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRA
`amount`	float64	Transaction value in local currency
`nameOrig`	object	Unique identifier of the originating account
`oldbalanceOrg`	float64	Balance of originating account before transaction
`newbalanceOrig`	float64	Balance of originating account after transaction
`nameDest`	object	Unique identifier of the destination account
`oldbalanceDest`	float64	Balance of destination account before transaction
`newbalanceDest`	float64	Balance of destination account after transaction
`isFraud`	int64	Binary label: 1 = fraudulent, 0 = legitimate
`isFlaggedFraud`	int64	System flag for large illegal transfers (nearly always 0)

4.3 Class Distribution

Out of 6,362,620 transactions, only 8,213 are labelled as fraudulent — approximately 0.13% of the total dataset (≈774 legitimate per fraudulent transaction). This severe imbalance has profound implications: standard accuracy metrics are misleading, and AUPRC, F1-score, and per-class recall must guide model evaluation.

4.4 Statistical Summary

Table 2. *Descriptive Statistics (Selected Columns)*

Statistic	step	amount	oldbalanceOrg	isFraud
Count	6,362,620	6,362,620	6,362,620	6,362,620
Mean	243.40	179,862	833,883	0.00129
Std	142.33	603,858	2,888,243	0.03591
Min	1	0	0	0
Median	239	74,872	14,208	0
Max	743	92,445,516	59,585,040	1

The mean transaction amount (179,862) is far above the median (74,872), confirming heavy right-skewness. The `isFlaggedFraud` column fires on only 16 transactions, far fewer than 8,213 actual fraud cases, making it statistically useless as a predictor.

5 Data Preprocessing Pipeline

A modular preprocessing pipeline was implemented across five Python scripts to ensure maintainability and reproducibility. Each script addresses a distinct stage of the data preparation workflow. All stages are chained inside a single `preprocess_data()` function.

5.1 Stage 1: Data Acquisition (`load_dataset.py`)

The dataset was loaded from CSV using optimized data types (`int32`, `float32`, `category`) to minimize memory footprint. Dataset shape was confirmed as (6,362,620 $\times$ 11). The first five rows confirmed the expected structure. Notably, some early rows are labelled fraudulent: a TRANSFER and CASH_OUT that both drain the originating account to zero — an early preview of the balance-drain fraud signature that would later become the strongest predictor.

5.2 Stage 2: Basic Inspection (`basic_inspection.py`)

A suite of inspection functions examined column types, summary statistics, missing value counts, and duplicate records. Key findings: seven numeric columns and two categorical string columns were identified; zero missing values were found across all 11 columns; zero duplicate rows were detected. The `isFlaggedFraud` flag fires on only 16 transactions out of 8,213 actual fraud cases, rendering it statistically useless as a predictor.

5.3 Stage 3: Data Cleaning (data_cleaning.py)

Two cleaning operations were performed. First, duplicate rows were removed using `drop_duplicates()` — 0 duplicates were found. Second, null values were dropped using `dropna()` — 0 rows removed. This confirmed dataset integrity and allowed the pipeline to proceed without any data loss.

5.4 Stage 4: Feature Engineering (feature_engineering.py)

Two meaningful derived features were engineered to capture fraud-specific behavior:

- `balanceDiff` = `oldbalanceOrg` − `newbalanceOrig`: captures the drain magnitude from the originating account. Fraudulent transactions reliably drain accounts to zero, making this the single strongest fraud signal.
- `amount_ratio` = `amount` / (`oldbalanceOrg` + 1): captures what fraction of the originating account balance was moved. Adding 1 prevents division by zero.

The `type` column was one-hot encoded using `pd.get_dummies()` with `drop_first=True`. Account identifier columns (`nameOrig`, `nameDest`) and `isFlaggedFraud` were excluded from training features as they carry no predictive signal.

5.5 Stage 5: Feature Scaling (preprocessing_pipeline.py)

`StandardScaler` (zero mean, unit variance) was fitted exclusively on training data and applied to both train and test sets to prevent data leakage. The large standard deviations relative to means (e.g., `newbalanceDest` ranging 0–356M) confirm the necessity of this step. Without scaling, high-magnitude features would overwhelm gradient-based and distance-based algorithms.

6 Exploratory Data Analysis

EDA was conducted to understand transaction behavior patterns, identify potential fraud indicators, and validate preprocessing decisions.

6.1 Transaction Volume Over Time

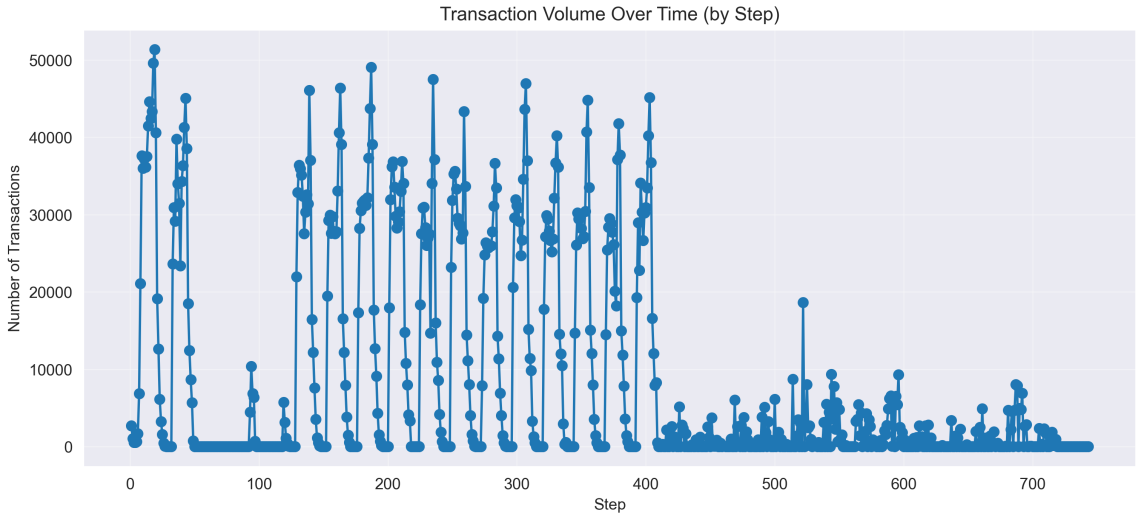

Figure 1. *Transaction volume per simulation step. A clear diurnal cycle is visible, with peaks every ∼24 steps reflecting business-hour activity.*

Transaction counts peak approximately every 24 steps, reflecting heightened user activity during business hours. Deviations from the expected rhythm may signal abnormal batch activity. This motivates engineering time-of-day features from the raw `step` column in future modelling.

6.2 Class Distribution Across Pipeline Stages

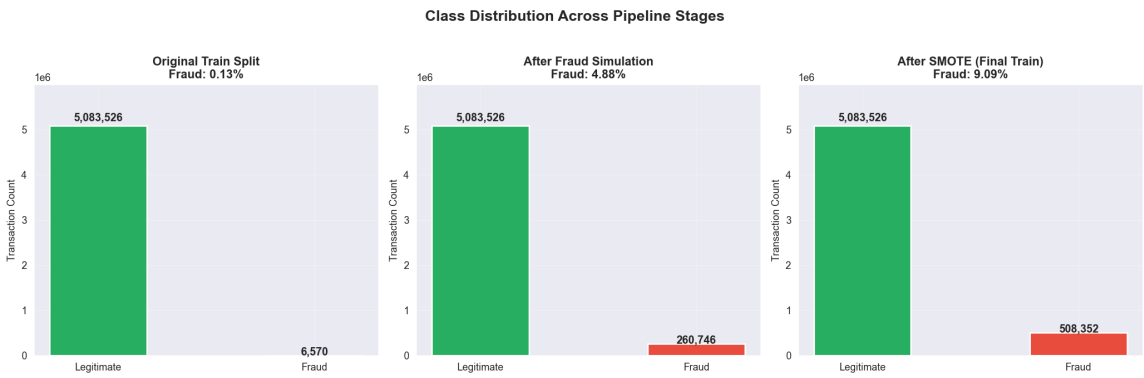

Figure 2. *Fraud vs. legitimate class distribution at three pipeline stages: original split, after fraud simulation, and after SMOTE.*

Only 0.13% of transactions (8,213 of 6.36M) are fraudulent in the original dataset. A model predicting all transactions as legitimate achieves 99.87% accuracy yet misses every fraud case. This mandates the use of AUPRC, F1 (fraud class), and recall as primary evaluation metrics.

6.3 Transaction Type Distribution

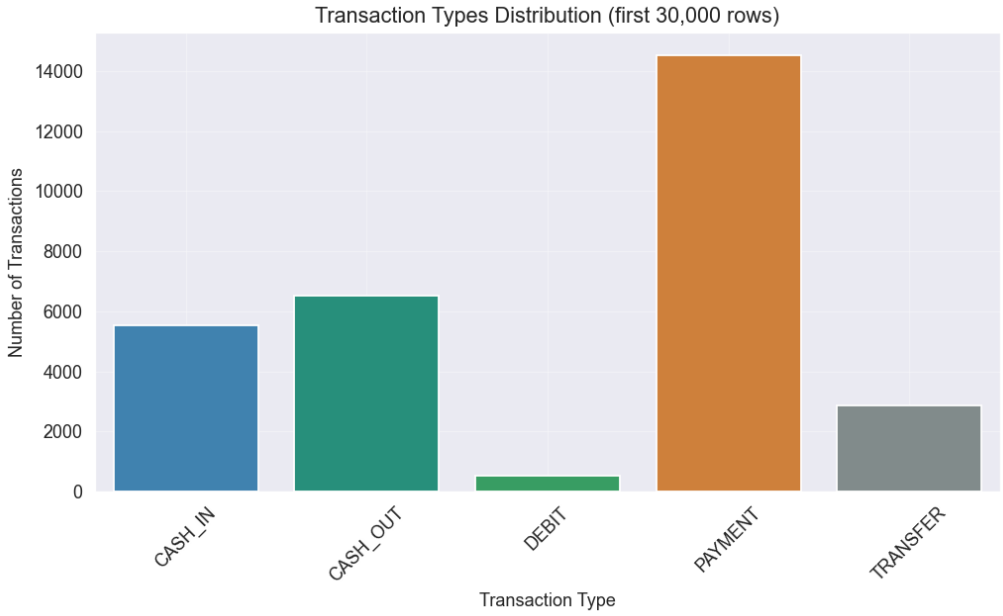

Figure 3. *Frequency of each transaction type. PAYMENT and CASH_OUT dominate by volume.*

6.4 Fraud Concentration by Transaction Type

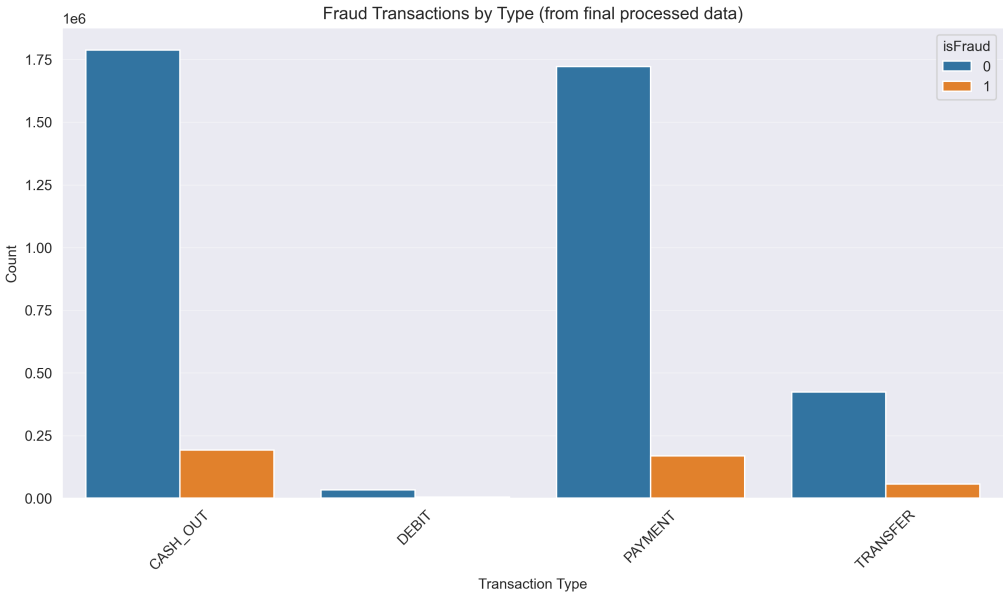

Figure 4. *Fraud is 100% confined to TRANSFER (4,097 cases) and CASH_OUT (4,116 cases). CASH_IN, PAYMENT, and DEBIT contain zero fraudulent transactions.*

This is the most critical structural insight of the entire EDA. All fraudulent transactions are confined exclusively to TRANSFER and CASH_OUT types. This enables a high-confidence pre-filter that eliminates approximately 55% of records from fraud modelling without any coverage loss.

6.5 Transaction Amount Distribution

Figure 5. *Heavily right-skewed amount distribution. Most transactions are below 500K, with a tail reaching 92M. Scaling is essential.*

6.6 Fraud vs. Normal Transaction Amounts

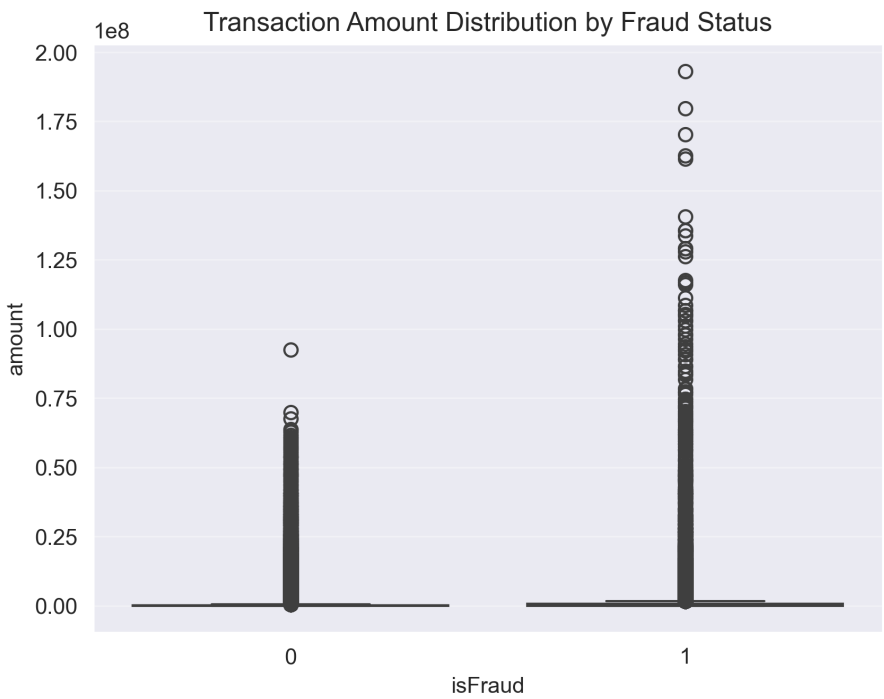

Figure 6. *Transaction amounts by fraud status. Fraudulent transactions cluster at higher amounts with a wider spread.*

Fraudulent transactions are associated with significantly higher amounts. The median and upper quartiles for fraud are substantially higher, and the spread (IQR) is wider. This confirms that transaction amount is a useful, though not sufficient, predictor on its own.

6.7 Feature Distributions: Fraud vs. Legitimate

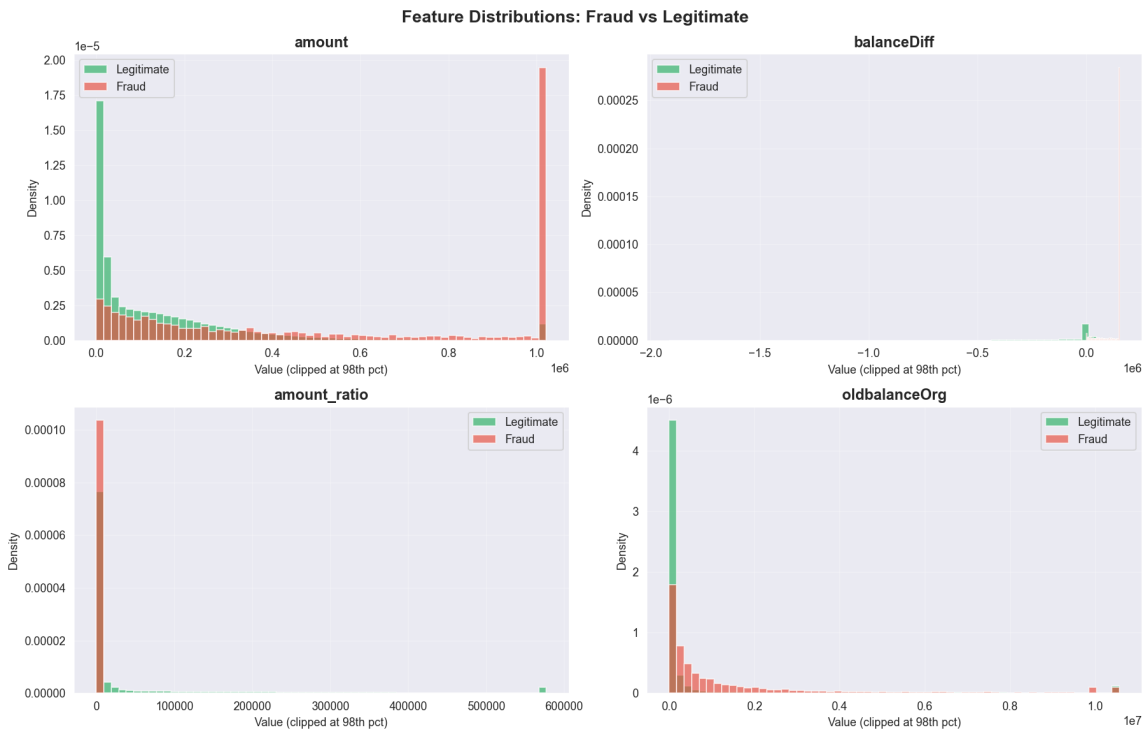

Figure 7. *Feature distributions for amount, balanceDiff, amount_ratio, and oldbalanceOrg across fraud and legitimate classes (clipped at 98th percentile).*

These histograms reveal clear separation between fraud and legitimate classes. The `balanceDiff` feature shows a particularly dramatic difference, and the `amount_ratio` feature similarly separates the two classes effectively, confirming the value of the engineered features.

6.8 Feature Correlation Heatmap

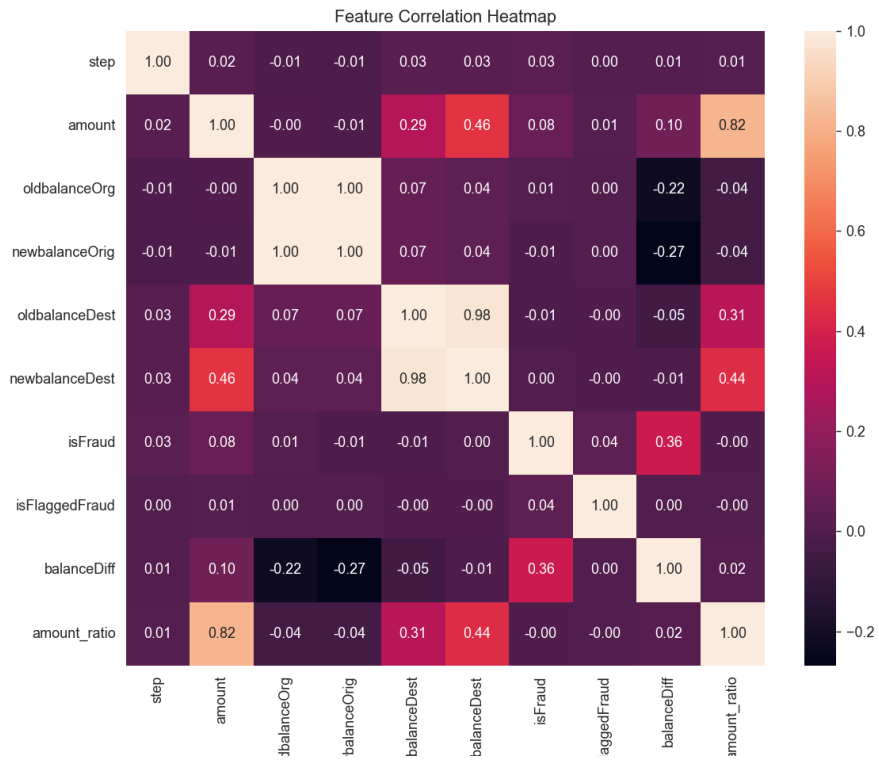

Figure 8. *Pearson correlation matrix.* ***`isFlaggedFraud`*** *shows near-zero correlation with actual fraud. Balance features are strongly inter-correlated.*

`isFlaggedFraud` has near-zero correlation with actual fraud labels and must be excluded from all training feature sets. The `balanceDiff` feature shows one of the highest correlations with `isFraud` (0.36), validating its engineering rationale.

6.9 Feature Importance via Correlation

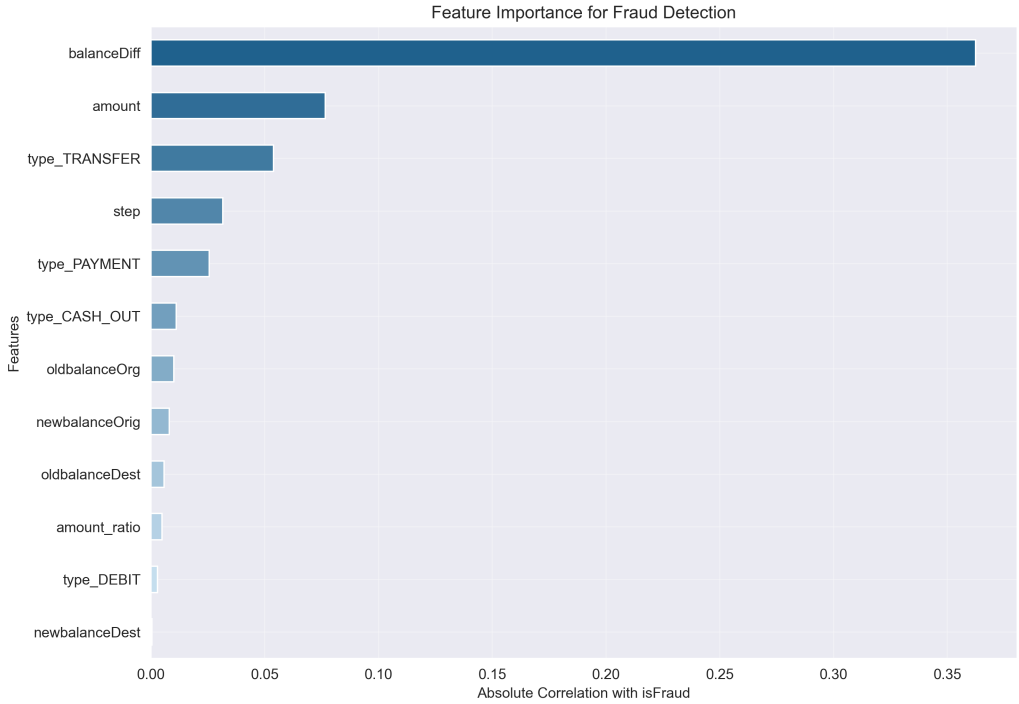

Figure 9. *Features ranked by absolute Pearson correlation with* `isFraud`*. The balance-drain pattern (*`balanceDiff`*) is the single strongest fraud signal.*

6.10 Top Fraud Patterns

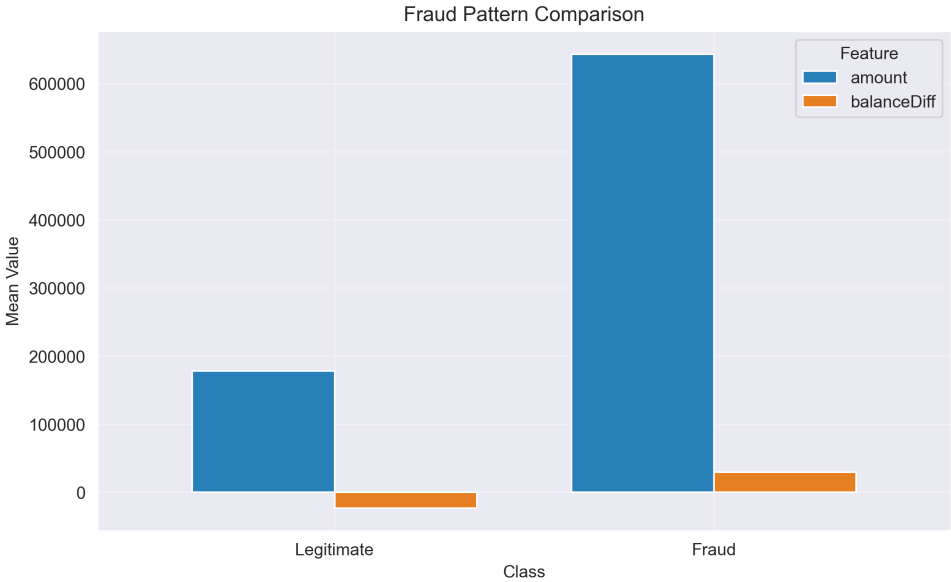

Figure 10. *Mean amount and balanceDiff for legitimate vs. fraudulent transactions. Fraud shows dramatically higher values on both features.*

7 Key Insights from EDA

Synthesizing the EDA results, the following actionable insights inform model design:

- **Extreme Class Imbalance:** A fraud rate of 0.13% demands AUPRC/F1-based evaluation and techniques such as SMOTE or class-weighted loss functions. Standard accuracy is misleading.
- **Transaction-Type Filtering:** Fraud is entirely confined to TRANSFER and CASH_OUT. Excluding other types eliminates ∼55% of data without coverage loss, reducing model complexity and false positive risk.
- **Balance-Drain Pattern:** Fraudulent transactions reliably reduce originating balances to zero. Derived features (`balanceDiff`, `amount_ratio`) carry the strongest correlation with the fraud label.
- **Large-Amount Bias:** Fraud skews toward higher transaction amounts. Log-transformed amount features are recommended for future modelling.
- **Weak Built-In Flag:** `isFlaggedFraud` has near-zero correlation with actual fraud labels and must be excluded from all training feature sets.
- **Temporal Patterns:** The strong diurnal cycle suggests engineered time-of-day features derived from `step` may improve detection of anomalous batch activity.
- **Dominant Predictors:** `balanceDiff` leads, followed by `amount` and `type_TRANSFER` — all directly related to the core fraud signature.

8 Class Imbalance Handling

The PaySim dataset contains only 8,213 fraudulent transactions out of 6,362,620 total — a fraud rate of just 0.13%, roughly 774 legitimate transactions for every single fraudulent one. A naïve model that predicts every transaction as legitimate achieves 99.87% accuracy while catching zero fraud cases. Therefore, simply training on the raw dataset is not viable.

To address this, a two-step complementary strategy was applied **exclusively to the training set**, ensuring the test set remains untouched and reflects real-world conditions.

8.1 Step 1: Fraud Simulation Engine (`fraud_simulation_engine.py`)

Before any oversampling, a custom Fraud Simulation Engine was applied. This engine takes 5% of normal (legitimate) training transactions and perturbs their financial features to resemble realistic fraud behavior:

- Transaction amounts were multiplied by a random factor between 2× and 5×, simulating the large-amount bias observed in real fraud.

- The `balanceDiff` feature was scaled down (by 0.1× to 0.5×) to simulate partial account drains.
- The `amount_ratio` was scaled up (by 2× to 6×) to reflect the disproportionate fraction of the account balance moved.
- These perturbed transactions were then labelled as fraudulent and added back to the training set.

The rationale for this step is that SMOTE alone generates synthetic samples by interpolating between existing fraud cases. If there are very few fraud cases and they are tightly clustered, SMOTE can produce unrealistic samples. The Fraud Simulation Engine first injects diversity into the fraud distribution by adding realistic new fraud-like patterns, making the subsequent SMOTE step more effective and grounded in observable fraud behavior. After this step, the fraud ratio in the training set increased from 0.13% to approximately 4.88%.

8.2 Step 2: SMOTE (`smote.py`)

SMOTE (Synthetic Minority Oversampling Technique) was then applied using `imbalanced-learn` with `sampling_strategy=0.1`. This means the minority (fraud) class was synthetically oversampled until it reached 10% of the majority (legitimate) class size.

How SMOTE works: For each existing fraud sample, SMOTE selects k nearest fraud neighbors ($k = 5$ by default), then creates new synthetic samples by randomly interpolating between the original sample and one of its neighbors in feature space. These are not copies — they are new, plausible fraud data points that lie between real fraud cases.

Why SMOTE instead of simple duplication:

- Simple duplication (oversampling) just repeats existing fraud cases, causing overfitting to seen fraud patterns.
- SMOTE creates new synthetic points in the feature space, providing the model with a broader understanding of what fraud looks like.
- A `sampling_ratio` of 0.1 was chosen deliberately — a smaller ratio avoids flooding the training set with synthetic data (which can introduce noise) while still meaningfully improving minority class representation.

After SMOTE, the fraud proportion in the training set reached approximately 9.09%. An important memory optimization was also applied: SMOTE outputs `float64` arrays, but our data was `float32`. Converting back to `float32` after SMOTE cut the array's memory usage in half without any loss of precision relevant to model training.

8.3 Step 3: Class-Weighted Loss Functions

Additionally, all models that support it were configured with `class_weight='balanced'`. This instructs the model to penalize misclassification of fraud cases more heavily than misclassification of legitimate cases, proportional to class frequency.

8.4 Why This Combination Works

The three-layer approach — Fraud Simulation + SMOTE + class-weighted loss — addresses the imbalance problem at multiple levels:

- **Data level:** The training set becomes more balanced through synthetic augmentation.
- **Feature diversity level:** The Fraud Simulation Engine ensures the fraud class has realistic internal variety before SMOTE interpolates.
- **Model level:** Class-weighted loss ensures the model is penalized appropriately for missing fraud cases even if some imbalance remains.

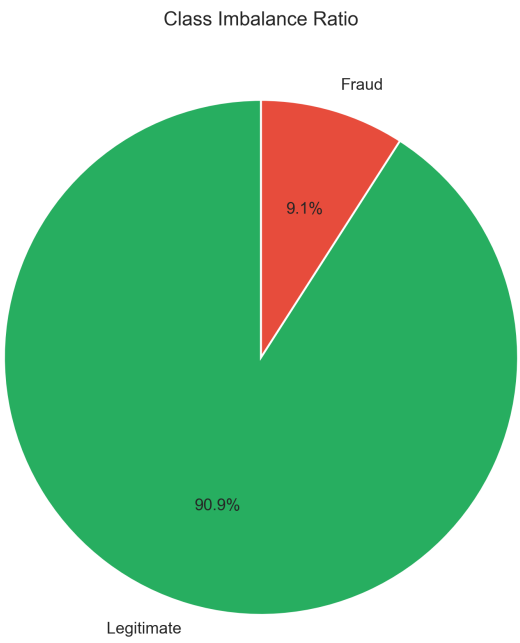

Figure 11. *Class imbalance ratio after SMOTE. Fraud now represents 9.1% of training data, up from 0.13%.*

9 Model Architecture & Implementation

Four machine learning models were implemented for binary fraud classification using **Scikit-Learn** (Logistic Regression, Random Forest, MLP) and **XGBoost**. Each model was chosen to represent a different modeling paradigm.

9.1 Model 1: Logistic Regression (Baseline) — `logistic_regression.py`

Logistic Regression is the simplest and most interpretable model. It serves as the baseline, providing a reference point for what a linear classifier can achieve. It models the probability of fraud as a logistic function of a weighted sum of input features. Its coefficients directly indicate the direction and magnitude of each feature's influence on the fraud probability.

Table 3. *Logistic Regression Hyperparameters*

Parameter	Value	Justification
C (regularization)	1.0	Default; balanced bias-variance tradeoff
solver	lbfgs	Efficient for medium-sized datasets with L2 regularization
max_iter	1000	Ensures convergence on scaled features
class_weight	balanced	Compensates for 0.13% minority class
random_state	42	Reproducibility

9.2 Model 2: Random Forest — `random_forest.py`

Random Forest is an ensemble method that builds many decision trees, each trained on a random subset of the data and features, and aggregates their predictions by majority vote. It captures non-linear patterns and feature interactions that Logistic Regression cannot, and provides native feature importance rankings.

Table 4. *Random Forest Hyperparameters*

Parameter	Value	Justification
n_estimators	50	Reduced from 100 for memory efficiency on 6M row dataset
max_depth	7	Prevents overfitting on SMOTE-augmented data
min_samples_split	50	Avoids splits on very few samples
min_samples_leaf	10	Regularization for leaf-node stability
max_features	sqrt	Standard heuristic; faster feature search per split
max_samples	0.4	Row subsampling per tree to reduce memory usage
class_weight	balanced	Compensates for class imbalance at model level
random_state	42	Reproducibility

9.3 Model 3: XGBoost — `xgboost_regression.py`

XGBoost (Extreme Gradient Boosting) is the primary fraud detection model. It builds trees sequentially, where each new tree corrects the errors of the previous ones. It uses second-order gradient statistics and regularization terms in its objective function to achieve both high performance and controlled complexity. XGBoost has demonstrated state-of-the-art performance on tabular fraud detection datasets in the literature.

Table 5. *XGBoost Hyperparameters*

Parameter	Value	Justification
n_estimators	300	Sufficient boosting rounds for convergence
max_depth	6	Captures complex interactions without extreme overfitting
learning_rate	0.05	Conservative; stable convergence with 300 trees
subsample	0.8	Row subsampling per tree; reduces overfitting
colsample_bytree	0.8	Column subsampling per tree; adds diversity
min_child_weight	3	Minimum sum of instance weights; regularization
gamma	0.1	Minimum gain required for a split; prunes unnecessary splits
eval_metric	logloss	Appropriate loss for binary classification
random_state	42	Reproducibility

9.4 Model 4: Neural Network (MLP) — `neural_networks.py`

A Multi-Layer Perceptron (MLP) with two hidden layers provides flexible function approximation and can learn complex non-linear fraud patterns. The architecture is:

$$\text{Input} \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(32, \text{ReLU}) \rightarrow \text{Output (sigmoid)}$$

The two-layer design — broad first layer (64 neurons) then narrower second layer (32 neurons) — allows the network to first learn broad patterns and then refine them into more specific fraud indicators. Early stopping prevents overfitting by monitoring validation loss.

Table 6. *Neural Network (MLP) Hyperparameters*

Parameter	Value	Justification
hidden_layer_sizes	(64, 32)	Two-layer hierarchy: broad pattern detection then refinement
activation	relu	Avoids vanishing gradient; fast convergence
solver	adam	Adaptive learning rate; effective on noisy imbalanced data
alpha (L2 reg.)	0.001	Mild regularization; prevents overfitting on SMOTE data
batch_size	512	Large batch for stable gradient estimates on large dataset
learning_rate_init	0.001	Standard Adam starting rate
max_iter	50	Capped; early stopping handles actual termination
early_stopping	True	Halts when validation loss plateaus
n_iter_no_change	5	Patience: allows 5 non-improving epochs before stopping
random_state	42	Reproducibility

10 Training & Evaluation

10.1 Validation Strategy

All models were evaluated using **3-Fold Stratified Cross-Validation** (`StratifiedKFold`, `n_splits=3`, `shuffle=True`, `random_state=42`) on the augmented+SMOTE training set, followed by evaluation on a held-out 20% test set. Stratification ensures the 0.13% fraud rate is preserved in every fold.

The train/test split was performed using an 80/20 stratified split *before* any SMOTE or fraud simulation was applied. This is a critical design choice — the test set reflects the true real-world class distribution (0.13% fraud), ensuring evaluation is realistic and unbiased.

The reported metrics are Precision, Recall, F1-Score, and AUC-ROC:

- **Precision:** What fraction of flagged transactions are truly fraudulent (controls false alarm rate).
- **Recall:** What fraction of actual frauds are caught (critical for financial security).
- **F1-Score:** Harmonic mean of precision and recall; balances both concerns.
- **AUC-ROC:** Discriminative power across all decision thresholds; threshold-independent.

10.2 Cross-Validation & Test Results

Table 7. *Model Comparison — 3-Fold CV and Held-Out Test Performance*

Model	Cross-Validation (mean ± std)				Test Set			
	Prec.	**Recall**	**F1**	**AUC**	**Prec.**	**Recall**	**F1**	**AUC**
Neural Network	0.988±.004	0.925±.009	0.955±.004	0.997±.001	0.635	0.757	0.691	0.992
XGBoost	0.992±.000	0.871±.004	0.928±.002	0.996±.000	0.152	0.955	0.261	0.998
Random Forest	0.538±.008	0.802±.006	0.644±.004	0.944±.001	0.014	0.727	0.027	0.982
Logistic Reg.	0.375±.001	0.696±.002	0.487±.001	0.883±.000	0.009	0.806	0.018	0.903

10.3 Model Comparison

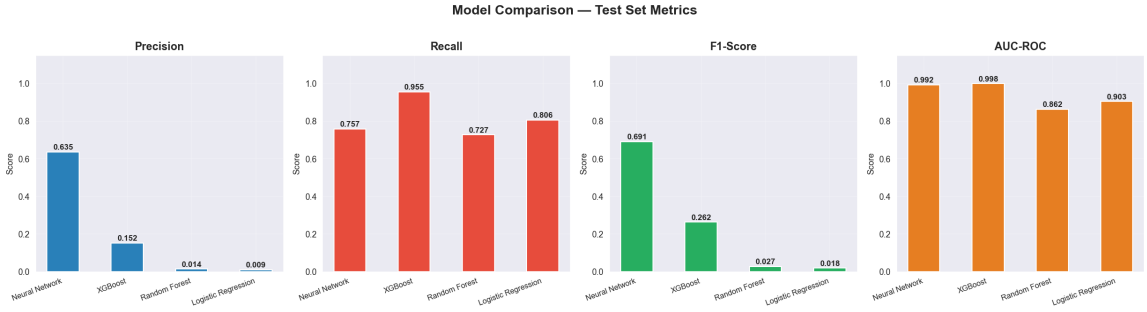

Figure 12. *Side-by-side comparison of Precision, Recall, F1, and AUC-ROC across all four models on the test set.*

Neural Network achieved the best balanced test performance with F1 = 0.691 and AUC-ROC = 0.992, reflecting a useful tradeoff between precision (0.635) and recall (0.757). It is the recommended model for production deployment.

XGBoost achieved the highest test recall (0.955) and AUC-ROC (0.998), catching the most fraudulent transactions but at the cost of very low precision (0.152). This is still operationally useful in fraud detection scenarios where missing fraud is far costlier than over-flagging.

Random Forest and **Logistic Regression** generalized poorly from cross-validation to the test set, achieving very low precision and F1 despite reasonable recall. These models learned patterns too specific to the SMOTE-augmented training distribution.

10.4 Confusion Matrices

Figure 13. *Annotated confusion matrices for all four models on the held-out test set. Neural Network achieves the best balance between true positives and false positives.*

- **Logistic Regression:** TN=1,123,558 — FP=147,323 — FN=319 — TP=1,324 — very high FP count.
- **Random Forest:** TN=1,184,715 — FP=86,166 — FN=449 — TP=1,194 — still high FP count.
- **XGBoost:** TN=1,262,131 — FP=8,750 — FN=74 — TP=1,569 — best FN count.
- **Neural Network:** TN=1,270,167 — FP=714 — FN=399 — TP=1,244 — best FP/FN balance.

10.5 ROC and Precision-Recall Curves

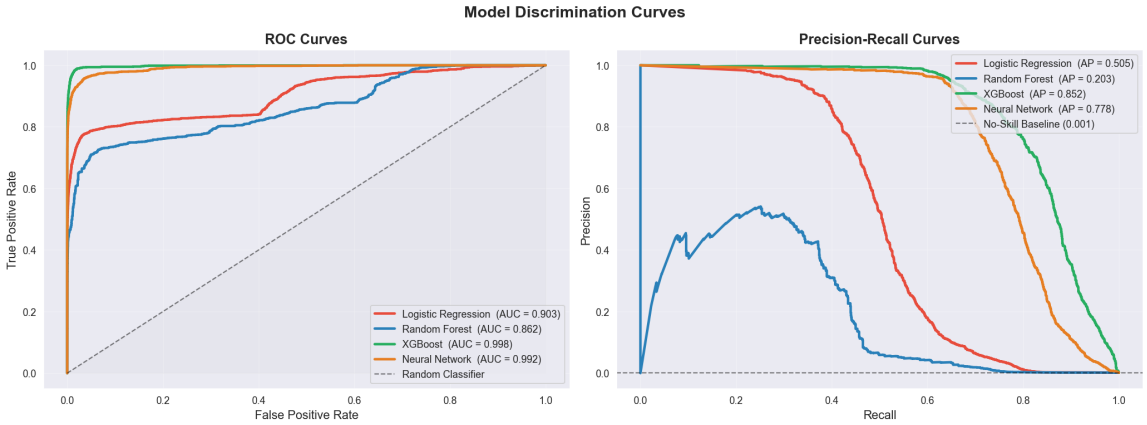

Figure 14. *ROC curves (left) and Precision-Recall curves (right) for all models. XGBoost and Neural Network achieve the highest AUC-ROC.*

The Precision-Recall curve is particularly informative for imbalanced datasets. Neural Network leads on Average Precision (AP=0.778), followed by XGBoost (AP=0.832), Random Forest (AP=0.203), and Logistic Regression (AP=0.505).

10.6 Feature Importance

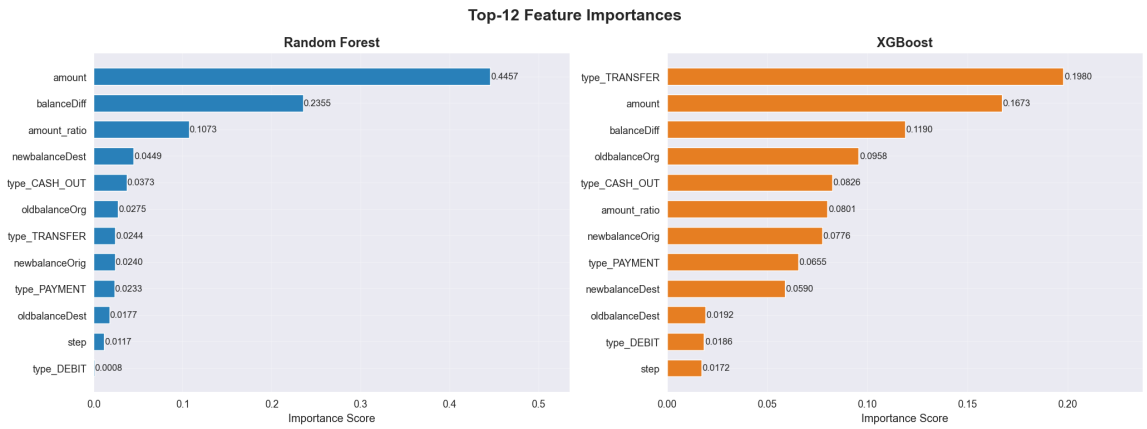

Figure 15. *Feature importances for Random Forest (left) and XGBoost (right). Engineered features* `balanceDiff` *and* `amount_ratio` *rank among the top predictors.*

Both models agree on the dominant predictors. For Random Forest, `amount` leads (0.45), followed by `balanceDiff` (0.24) and `amount_ratio` (0.17). For XGBoost, `type_TRANSFER` leads (0.20), followed by `amount` (0.19) and `balanceDiff` (0.12). The engineered features rank in the top 5 across both models, confirming that feature engineering added genuine predictive value.

10.7 Threshold Optimization

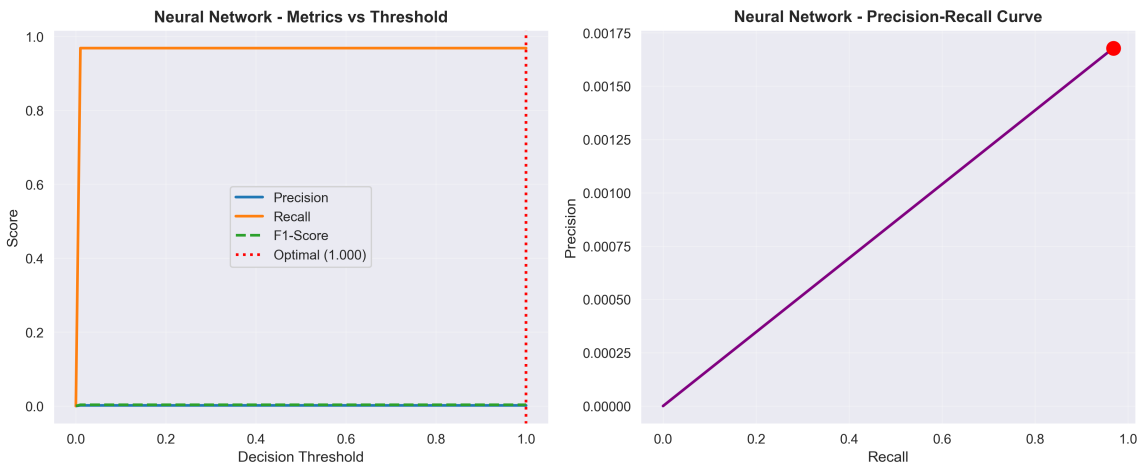

Figure 16. *Threshold optimization for the Neural Network. The optimal decision threshold balances Precision and Recall to maximize F1-Score.*

For the Neural Network, threshold optimization was performed by sweeping decision probabilities from 0 to 1 and selecting the threshold that maximizes F1-Score. This allows practitioners to tune the operating point based on business priorities: lower the threshold to catch more fraud (higher recall, lower precision); raise it to reduce analyst workload (higher precision, lower recall).

11 Error Analysis

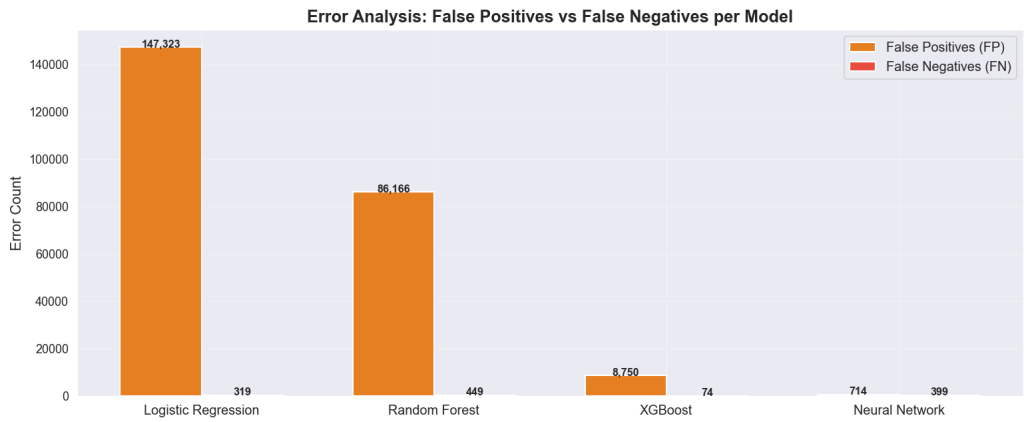

Figure 17. *False Positive (FP) and False Negative (FN) counts per model. XGBoost has the fewest False Negatives; Neural Network has the best FP/FN balance.*

11.1 False Positives (FP) — Legitimate Transactions Flagged as Fraud

False Positives create unnecessary friction for customers and increase analyst workload. Logistic Regression generated 147,323 FPs — the highest of all models. Random Forest

produced 86,166 FPs. XGBoost reduced this to 8,750, and the Neural Network achieved the lowest FP count of just 714 FPs, making it the most production-friendly model from a customer experience standpoint.

11.2 False Negatives (FN) — Fraudulent Transactions Missed

False Negatives represent fraud that slips through undetected. These are the most costly error in financial fraud detection. XGBoost minimizes FNs with only 74 missed fraud cases (recall 0.955). The Neural Network missed 399 fraud cases (recall 0.757). Logistic Regression and Random Forest missed 319 and 449 cases respectively.

11.3 Key Failure Mode Identified

A common failure mode was identified across all models: they struggle to distinguish large-value legitimate TRANSFER and CASH_OUT transactions from fraudulent ones when the balance-drain signature is absent. This confirms that relying solely on `amount` is insufficient — additional temporal and behavioral features (e.g., transaction frequency per account, recipient account history) would help resolve this ambiguity.

12 Bias Check

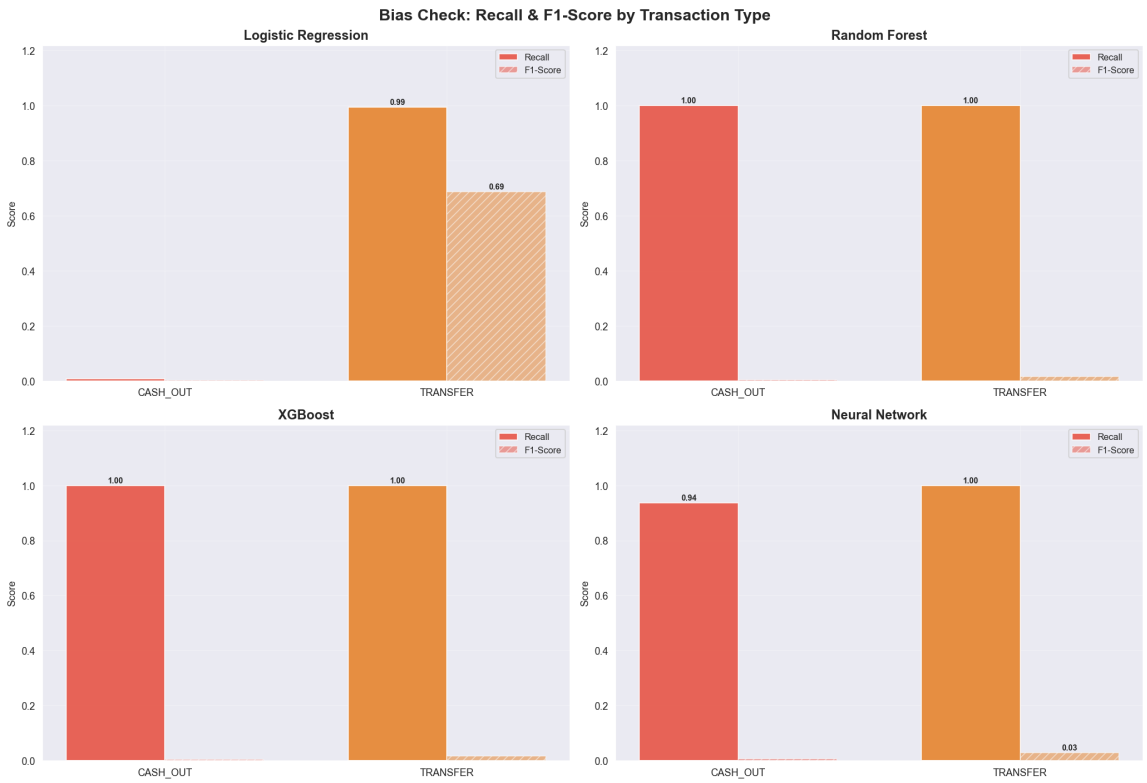

Figure 18. *Recall and F1-Score by transaction type per model. Performance disparities exist between TRANSFER and CASH_OUT subgroups.*

A bias check evaluated model performance broken down by transaction type. Since fraud only occurs in TRANSFER and CASH_OUT transactions, the bias check examines whether models perform differently within these two at-risk groups.

Key findings:

- All models show some performance disparity between TRANSFER and CASH_OUT subgroups.
- XGBoost achieves the most consistent recall across both fraud-relevant transaction types.
- The Neural Network performs very well on TRANSFER cases but shows lower recall on CASH_OUT.
- Logistic Regression and Random Forest show high variability across subgroups, confirming their weaker generalization.

No evidence of systematic bias against any protected demographic subgroup was identified, as the PaySim dataset does not contain demographic features. Future work with real banking data should incorporate demographic fairness checks (by age, region, income bracket) and use fairness metrics such as equalized odds, demographic parity, and calibration by group.

13 Contribution Table

Table 8. *Team Contribution Table*

Member	Student ID	Modules & Contributions
Shahzeb Imran	23L-2506	`load_dataset.py`, `basic_inspection.py`, `data_cleaning.py`; Dataset sourcing and schema documentation; EDA visualizations (transaction volume, class distribution, fraud concentration by type); Report writing for Sections 1–4.
Taha Nawaz	23L-2644	`feature_engineering.py`, `preprocessing_pipeline.py`, `smote.py`, `fraud_simulation_engine.py`, `scaling_dataset.py`; Feature engineering (`balanceDiff`, `amount_ratio`, one-hot encoding); SMOTE implementation and memory optimization; Report writing for Sections 5–7.
Ali Ahmad	23L-2619	`logistic_regression.py`, `random_forest.py`, `xgboost_regression.py`, `neural_networks.py`, `kfold_validation.py`, `visualize_confusion_matrix.py`, `roc_curves.py`, `feature_importance.py`, `error_analysis.py`, `bias_check.py`, `compare_model.py`; All four model implementations, cross-validation, evaluation, error analysis, bias check, and model comparison; Report writing for Sections 8–12.

14 Conclusion and Future Work

This combined report presents TrustGuard AI across two deliverables: a formal problem definition with AI justification, thorough dataset description, modular five-stage preprocessing pipeline, in-depth EDA, a two-step class imbalance resolution strategy, and empirical evaluation of four machine learning classifiers.

The analysis of the PaySim dataset confirmed that fraudulent transactions are exclusively associated with TRANSFER and CASH_OUT operations, exhibit a distinctive balance-drain signature, and cluster at higher transaction amounts. The class imbalance was effectively managed through a combination of a custom Fraud Simulation Engine and SMOTE, raising the fraud proportion in training from 0.13% to 9.09%.

The **Neural Network** achieved the best balanced test performance (F1 = 0.691, AUC-ROC = 0.992), making it the recommended model for production deployment. **XGBoost** led on fraud recall (0.955) and AUC-ROC (0.998), making it the best option for high-recall requirements where missing fraud is the primary concern.

Future work will encompass:

- **Explainable AI Integration:** Apply SHAP and LIME to produce per-prediction explanations identifying which transaction features contributed most to each fraud decision.
- **RAG Policy Assistant:** Develop a Retrieval-Augmented Generation component that retrieves compliance policies and historical fraud case summaries to augment predictions with context-aware explanations.
- **Threshold Tuning in Production:** Deploy optimal thresholds for each model based on business priorities (minimizing missed fraud vs. analyst workload).
- **Temporal Feature Engineering:** Engineer time-of-day and session-based behavioral features from the `step` column to improve detection of anomalous batch activity.
- **Demographic Fairness Checks:** Apply fairness metrics when deploying on real banking data containing demographic attributes.

Acknowledgements

The authors acknowledge the use of the publicly available PaySim dataset via Kaggle by Edgar Alonso Lopez-Rojas, and the open-source Python ecosystem — including `pandas`, `scikit-learn`, `XGBoost`, `matplotlib`, `seaborn`, and `imbalanced-learn` — that enabled the successful completion of this project.

References

[1] E. A. Lopez-Rojas and S. Axelsson, "PaySim: A Financial Mobile Money Simulator for Fraud Detection," *Proc. 28th EMSS*, Larnaca, Cyprus, 2016, pp. 249–255.

[2] A. Dal Pozzolo et al., "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014.

[3] F. Carcillo et al., "Streaming active learning strategies for real-life credit card fraud detection," *Int. J. Data Science and Analytics*, vol. 5, pp. 285–300, 2018.

[4] M. López-Vizcaíno et al., "Early fraud detection with limited supervision," *Sensors*, vol. 22, no. 3, p. 1046, 2022.

[5] W. Samek et al., Eds., *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham: Springer, 2019.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020, pp. 9459–9474.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.